

TAKE-HOME • ALTERNATIVE E

Incident-to-Patch Assistant

Claude reads recorded crash reports and suggests a fix. My code then tests that fix on a copy and decides whether it's accepted, without taking the model's word for anything.

 cLaude-opus-5-5

 Fixes are tested, not trusted

 Replays without an API key

From Crash to Tested Fix

The Bug

1 OF 5 FAILING

```
# quantity = 0  
ZeroDivisionError: integer division or modulo by zero
```

Given: a 4-line pricing function and 3 crash reports

Problem: a quantity of 0 crashes the division

Claude's Fix

5 OF 5 PASSING

```
+ if quantity == 0:  
+ return 0
```

Change: 2 lines, added after the input check

Result: all 5 reference cases pass, other 4 unchanged

How It Works

1

Group Crashes

Matching error and file become
one incident

2

Confirm Bug

Run tests first; call Claude only if a
test fails

3

Ask Claude

Returns diagnosis and patch as
JSON

4

Patch a Copy

The original baseline file is never
touched

5

Run Tests

Fixed test command with strict
timeout



Claude suggests, my code decides. A fix is accepted only if the tests show it works.

Making Sure the Answer Key Is Right



Locked Starting Files

The original files cannot change without the tool noticing. Each one has a recorded fingerprint.



Checked Answers Myself

I recalculated every expected value separately, without using the buggy code.



Extra Test Cases

Added 7 edge-case inputs, one more crash report, and one deliberately broken report.



Found a gap in the rules

For $(-1, 0)$ the rules say both 'return 0' and 'raise an error', and don't say which wins. I documented it rather than guessing. Claude also listed it as a risk in its answer.

Proving It Says No When It Should

NEGATIVE CONTROL

REJECTED

Broken Rounding

A fake fix that breaks rounding. The tests catch it, and the original code stays untouched.

DIFF CONTROL

REJECTED

Fix That Doesn't Fit

A fake fix that doesn't match the code is refused before anything runs.

CITATION GATE

REJECTED

Wrong Evidence

If Claude cites a crash report that doesn't exist or belongs to another bug, the run fails.



If no test reproduces the bug (for example EV3, which records correct behaviour), the tool stops before paying for a model call.

Where the AI Got It Wrong, and How I Caught It

runs/20261006T161408Z-ev1-live/report.html STATUS: CHECKED

Case	Args	Expected (frozen)	Baseline	Candidate	Change
zero-quantity target	[0, 0]	0	raises ZeroDivisionError fail	0 pass	changed
ordinary	[1000, 2]	500	500 pass	500 pass	same
rounding	[1001, 2]	501	501 pass	501 pass	same
zero-total	[0, 4]	0	0 pass	0 pass	same
single	[1999, 1]	1999	1999 pass	1999 pass	same

From the live run's report: the failing case now passes and the other four are unchanged, tested on a copy of the file.

Bug 1: wrong failure reason

A file-path mistake made a bad fix fail for the wrong reason. It was still reported as failed, never as a success. Fixed and covered by a test.

Bug 2: mixed-up bugs

I wrote a test to check one claim in my own README. It failed and showed that one incident could pick up another incident's failing test. Fixed and covered by a test.

Checks on every commit

16 automatic checks run before each commit, including code style, a scan for leaked API keys, the locked files, and all 28 tests.

What This Doesn't Prove

Only 12 inputs

Passing 12 test inputs shows the fix works for those cases. It isn't proof for every possible input.

Not a sandbox

The suggested code runs in a separate process with a time limit and is screened first, but that isn't real isolation.

One attempt

Claude gets one try. It doesn't retry after a failed test.

Answers vary

Asking again can give a different answer, so results are reproduced by replaying the saved response.

VERIFICATION

Try It Without an API Key

```
$ python -m assistant demo # replays the saved Claude response and grades the 5 required checks
$ python -m assistant replay runs/20261006T161408Z-ev1-live --recheck # reruns the tests on the saved fix
```

README.md

How it works, design choices, and limits

RESULTS.md

The 5 required checks: expected vs. actual

ai-workflow/

The AI tools and settings I used, and an example of catching a mistake